referenceSystem

Time systems, Earth orientation and coordinate transformations for satellite geodesy

Version	v2.1.209
Handover revision	1.209 — the patch component IS the revision (D107)
Build date	2026-08-21
Author	Matthias Weigelt
Developed with	Claude Opus 5 (Anthropic)
Standards	IERS Conventions (2010); IAU SOFA release 20231011; IAU 2006/2000A precession-nutation

*This manual is **generated** from the package itself. The version and build date are read from `toolboxVersion.m`, the layer structure from `Contents.m`, and every entry in the API chapter from the help block of the function it describes. A manual that disagreed with the toolbox could not be produced, which is the only kind of manual worth shipping.*

1 Theory

The toolbox realises one chain and everything else supports it: the rotation from the International Terrestrial Reference System to the Geocentric Celestial Reference System, at an arbitrary epoch, to an accuracy limited by the Earth orientation parameters rather than by the software.

1.1 The rotation chain

Following IERS Conventions (2010) Chapter 5 in the CIO-based formulation, the transformation factorises into four rotations, each with a distinct physical origin and a distinct error budget:

```
[GCRS] = Q(t) * R(t) * W(t) * [ITRS]

W(t)  polar motion       xp, yp and the TIO locator s'
R(t)  Earth rotation     ERA, linear in UT1
Q(t)  precession-nutation  CIP X, Y and the CIO locator s
```

The split matters operationally as well as physically. **Q varies on a scale of hours, W and UT1 are tabulated daily, and only the Earth rotation angle is fast** — and it is linear in UT1, hence essentially free. That asymmetry is what makes the interpolated mode possible at all: everything expensive is either constant over an arc or smooth in time.

1.2 Which error dominates

Sub-millimetre accuracy at the Earth's surface corresponds to about **32 microarcseconds** of rotation. The toolbox's own contribution is far inside that; the Earth orientation parameters are not:

Limit on the absolute rotation	Magnitude	At the surface
UT1 uncertainty (~5 µs)	3.6e-10 rad	~2 mm
dX/dY uncertainty (~50 µas)	2.4e-10 rad	~1.5 mm
xp/yp uncertainty (~30 µas)	1.5e-10 rad	~1 mm
toolbox, direct path	~1e-16 rad	**~0.6 nm**
toolbox, interpolated path	≤ 2.4e-12 rad	**~0.015 mm**

Read that table before choosing a tolerance anywhere. **Sub-millimetre absolute accuracy is not achievable from today's EOP series**, and no amount of care in the software changes it. What the software must do is stay out of the way — and remain out of the way when the EOP improve, which is why every tolerance in the test suite asserts *algorithmic* agreement and never EOP-level agreement.

1.3 Time scales

Eleven scales are supported and the conversions among them are rigorous, not nominal, wherever a rigorous definition exists. UTC is the awkward one: it is not a uniform scale, a leap second makes one day 86401 seconds long, and an epoch inside that second is representable only if the day and the second-of-day are carried separately. That is why the `epoch` type is a two-field struct and never a serial date number.

Relation	Nature
TAI – UTC	Integer leap seconds, from a tabulated authority
TT = TAI + 32.184 s	Exact by definition
TCG, TCB	Exact linear rate relations (IAU 2000 B1.9, B1.5)
TDB – TT	Periodic series; SOFA iauDtdb, better than 1e-5 s against the Almanac formula
UT1 – UTC	**Measured**, from the EOP series. Not a conversion but an observation

GPS, GST, BDT	Nominal integer offsets from TAI. Documented as nominal, **not realised**

1.4 What is deliberately approximate

Every approximation the toolbox makes is named and quantified in the Accuracy chapter and in the browsable ACCURACY block of the function that makes it. Three are worth stating here because they shape how the toolbox is used:

- **Derivatives are differenced, not differentiated.** Rdot and Rddot come from central differences with a measured optimum step of 200 s. Analytic differentiation would mean duplicating SOFA's nutation tables, and every step from 1 s to 1800 s meets a 1 nm/s velocity budget, so the choice is not critical.
- **The interpolated mode fits the exact model, it does not replace it.** Chebyshev panels are built from the same SOFA evaluation the direct path runs; the fit error is bounded at fractions of a microarcsecond and asserted against the direct chain.
- **Sub-daily EOP corrections are on by default.** Ocean tide and libration reach ~0.5 mas in polar motion and ~50 µs in UT1, which is ~15 mm. They are not optional for GRACE-class work.

2 Implementation philosophy

The design rules below are not style preferences. Each was adopted after a specific defect, and the defect is recorded with it in the handover document. They are repeated here because a user who knows why the interfaces look as they do can predict what they will do.

Correctness is checked against something outside the toolbox

A self-test cannot catch a consistently wrong convention: a mistaken Euler sense still produces an orthogonal, unit-determinant matrix that round-trips through its own inverse. Conventions are therefore checked against NAIF SPICE, PROJ, JPL HORIZONS, published worked examples and real GRACE-FO flight data.

A tolerance is a claim about which error dominates

When a precision test fails, the residual is decomposed into its physical contributors before any bound is touched. Loosening a tolerance to make a test pass usually destroys exactly the discrimination the test existed for.

Metadata travels with the data

The CIP model reference, the attitude convention, the ephemeris generation and the leap-second source all travel in a meta field rather than being assumed. The worst defect inherited from v1 was pole offsets referred to an assumed precession-nutation model: 4.2 mm at 2025, and invisible in the result.

Measured, never declared

Where the toolbox can ask, it asks. Kernel pairing is read from the SPK comment area, not from the filename. The MEX build reports what the compiler actually did, not what it was asked to do.

One authority per fact

The version lives in one file, the layer manifest in one file, the leap-second table behind one function. A consumer that re-derives a fact by parsing another consumer has made itself a second authority, and the two will disagree on the day nobody is looking.

Refuse, or report — never fall back silently

A guard that cannot determine a fault proceeds. A function that cannot verify a claim says so in its returned metadata and in a warning. Nothing degrades quietly to a plausible answer.

The artefact must survive the failure of what produces it

A run that dies still writes its report, marked incomplete, with the exception that killed it. A partial download returns its manifest. Three separate defects in this project were a preserving mechanism destroying what it existed to preserve.

2.1 Layer architecture

Calls go downward only, and the layering is asserted by a test that reads the manifest in `Contents.m`. A function that calls upward is a defect the suite reports by name.

```
L6  dispatch, build, documentation, tests
L5  orbit, attitude, navigation, ephemerides, body frames
L4  coordinate representations and rotation algebra
L3  terrestrial frame realizations (ITRF, datums)
L2  Earth orientation transformation (ITRS <-> GCRS)
L1  Earth orientation parameters
L0  time scales and constants
C   MEX gateway over IAU SOFA and the IERS routines
```

3 Installation

One command does everything and reports what it did. It never repairs silently: anything it cannot do is named, with the reason.

```
>> cd <folder containing referenceSystem>
>> addpath referenceSystem
>> status = setupReferenceSystem('Verbose', true)

   ... path set, MEX built, documentation registered ...
   referenceSystem 2.1.209 is ready.
```

The setup puts the distribution folder on the path (never `+referenceSystem`, which is a namespace folder and must not be), removes a second installation if it finds one, compiles the MEX gateway against the bundled SOFA and IERS sources, registers the documentation, and reports the leap-second horizon.

3.1 External data

What	Needed for	How
EOP series	Everything. There is no built-in default and no zero fallback	Fetched by setup. Alone: `setupReferenceSystem('Fetch', "eop")`
SPICE kernels	The `"mice"` ephemeris tier and PCK body frames	Fetched by setup. To see the bill first, `downloadKernels('Mode', "list")`
MICE toolkit	The same	Detected, never installed. It ships as platform-specific compiled MEX
Geoid grid	Orthometric heights	Not bundled, for licensing and size reasons

Nothing is fetched by default: setup installs the toolbox and leaves the data alone. Pass `'Fetch', "all"` to install it, roughly 700 MB on a first run and nothing at all afterwards, because a file already on disk is skipped; `'KernelSet', "minimal"` leaves out the 650 MB small-body kernel. Where it goes is `referenceSystem.dataFolder`, which remembers the choice. The exploratory call remains the harmless one: `downloadKernels()` returns the plan and touches neither network nor disk.

3.2 Checking the installation

```
>> referenceSystem.mexInfo()          % gateway, SOFA release, OpenMP
>> referenceSystem.toolboxVersion()   % version, author, provenance
>> referenceSystem.runTestSuite()     % full suite + written report
```

The report is written to the **toolbox root**, not to the current folder, and its header states the version that produced it. Both properties exist because a report that follows the working directory leaves a stale one where people look, and a stale report is indistinguishable from a fresh one.

4 User guidelines

4.1 The epoch type

Every function that takes a time takes an `epoch` struct: an integer day, a second-of-day, and a scale name. Never a datenum, never a scalar of seconds. The reason is UTC: a leap second makes one day 86401 s long, and only a split representation can name an instant inside it.

```
utc = referenceSystem.daySecondToEpoch([57754 43200], "UTC");
utc = referenceSystem.mjdToEpoch(57754.5, "UTC");
utc = referenceSystem.datetimeToEpoch(datetime(2017,1,1,12,0,0), "UTC");
tt  = referenceSystem.convertTimeScale(utc, "TT");
```

4.2 Choosing a performance mode

Mode	Use for	Interface
direct	Everything except hot loops. The accuracy reference	`rotationItrsToGcrs(t, 'Eop', eop)`
interpolated	MATLAB-level loops and long arcs	`earthOrientationInterpolant(span, 'Eop', eop)` then pass it as `'Interpolant'`
C API	Real orbit integration, measured at 0.564 µs per call	link `rsApi.h` from your own MEX

MATLAB's own call and validation overhead is 5–15 µs and cannot be optimised away from inside MATLAB. There is deliberately no MATLAB single-call performance target: a target below the language's floor is not a target. If an integrator needs better, the integrator must be in C — which is why the C API is a deliverable and not an afterthought.

4.3 Things that will bite

- **Partials are per radian, not per arcsecond.** Polar motion is published in arcseconds; it reaches these functions in radians. A factor of 206265 in a Jacobian is invisible in the answer — it just makes the estimate converge to the wrong place. `info.polarMotionUnits` says so on every call.
- **The zonal tide correction is refused without an opt-in.** A C04 series already contains those terms as observed values, so applying them again is invisible and costs metres.
- **The lunar body-fixed frame never defaults.** MOON_ME and MOON_PA differ by about 860 m at the surface. The caller names one.
- **The bundled ephemeris is kilometres, not millimetres.** The 1 mm BCRS transformation is millimetre-accurate *as a correction*, applied to a position whose own error is four orders larger. Do not combine them and quote the smaller number.
- **Mixed ephemeris generations warn, they do not fail.** SPICE resolves an overlapping body from whichever kernel was furnished last, so an unintended mixture is silent in the result. Read `meta.generations`.

5 Step-by-step guides

5.1 A GRACE-like arc: ECEF to ECI with velocities

```
% 1. Read the Earth orientation series once per session.
eop = referenceSystem.readEop("eopc04_20.1962-now");

% 2. Build the epoch column for the whole arc, in one call.
day = repmat(57754, 5000, 1);
sec = (0:4999).' * 10;
utc = referenceSystem.daySecondToEpoch([day sec], "UTC");

% 3. One rotation call for the arc. Rdot comes free of a second
%    pass: the three sample times per epoch are interpolated once.
[R, Rdot, ~, info] = referenceSystem.rotationItrsToGcrs(utc, ...
    'Eop', eop);

% 4. Apply it to position and velocity together.
[rGcrs, vGcrs] = referenceSystem.applyRotation(R, Rdot, rItrs, vItrs);

assert(info.path == "direct");
```

5.2 The same arc, interpolated

```
% Build the panels once for the span, then never touch the series.
span = referenceSystem.daySecondToEpoch([57754 0; 57757 0], "UTC");
eoi  = referenceSystem.earthOrientationInterpolant(span, 'Eop', eop);

% No 'Eop' is passed: a seven-channel interpolant carries every
% quantity the rotation needs, including the excess length of day.
[R, Rdot, ~, info] = referenceSystem.rotationItrsToGcrs(utc, ...
    'Interpolant', eoi);

assert(info.path == "interpolated");
assert(info.sampleSource == "panels");   % nothing read the series
```

5.3 An integrator hot loop, in C

```
/* MATLAB writes the panels; C reads the file MATLAB wrote. */
referenceSystem.writeInterpolant(eoi, "arc.eopi");   % in MATLAB

/* Two-call protocol: header first, then sized arrays. */
rsApiInterpolant ip;
rsApiReadInterpolant("arc.eopi", &ip, NULL, 0, NULL, 0);
coefficient = malloc(rsApiInterpolantSize(&ip) * sizeof(double));
taiMinusUtc = malloc(ip.panelCount * sizeof(double));
rsApiReadInterpolant("arc.eopi", &ip, coefficient,
                     rsApiInterpolantSize(&ip),
                     taiMinusUtc, ip.panelCount);

double R[9];
rsApiRotation(&ip, mjdUtc, R);      /* 0.564 us per call */
```

If you build an `rsApiInterpolant` by hand rather than reading a file, set `quantityCount`. Zero means six, so code written before v2.1.179 keeps its meaning.

5.4 Estimating EOP corrections

```
[dRdXp, dRdYp, dRdDut1, info] = ...
    referenceSystem.rotationItrsToGcrsPartials(utc, 'Eop', eop);

% Scale to the parameterisation your normal matrix uses.
perArcsecond = referenceSystem.iersConstants().arcsecondsToRadians;
dRdXpArcsec  = dRdXp * perArcsecond;
```

6 Accuracy and validation

Every figure below is **measured**. Where a figure was asserted and later measured, the measurement replaced it and the correction is recorded in the handover revision log.

6.1 The rotation chain

Check	Against	Result
CIP X, Y	SOFA cookbook	9.0e-17, 2.9e-16
Full ITRS→GCRS matrix	SOFA cookbook	2.6e-15 (5e-4 µas)
ERA	SOFA cookbook	5.9e-13 deg
Equinox route vs CIO route	each other, swept over epochs	4.4e-16 rad = 9.1e-5 µas
Orthogonality ■R'R – I■	—	1.1e-16
Skew invariant	—	4.2e-19 rad/s
Interpolated vs direct	each other, across panel boundaries	< 5e-12 on a direction cosine

6.2 Against outside authorities

Domain	Authority	Result
Geodetic ↔ Cartesian, Helmert, ITRF2020 chain	PROJ 9.5.1	bit for bit; 0.0 m; 1e-9 m
Euler sequences and quaternion products	NAIF SPICE	1e-14 over 9600 comparisons
WGCCRE Table 1 pole and rate	orbital poles from the ephemeris; published periods	0.011–0.034 deg; 2.5e-6–8.8e-11
WGCCRE Table 1 transcription	NAIF PCK, independently encoded	2e-6 in matrix elements ≈ 0.4 arcsec
Planetary ephemeris, "mice" tier	JPL HORIZONS fixture	< 1000 m (DE440 against DE441)
Small-body path	JPL HORIZONS fixture	8021.9 m for Ceres — 2e-8 of 2.77 au
Attitude interpolation	held-out GRACE-FO flight data	4.4e-6 rad at 2 s gaps
Fundamental arguments	ported IERS FUNDARG.F	9.8e-12 rad = 2.0 µas

*The last row is the shape of an honest disagreement. The supplied Fortran truncates two constants and does not reproduce the test case in its own header; it is **pinned and measured rather than repaired**, because repairing it would break the byte-intactness under which the other five routines are validated.*

6.3 Convergence and derivatives

Rdot uses central differences at a measured optimum of 200 s, order 2.14 confirmed from 600 to 300 s, with a roundoff plateau below about 150 s. **Every step from 1 s to 1800 s meets a 1 nm/s velocity and 1e-10 m/s² acceleration budget**, so the choice is not critical — the opposite of what an earlier unmeasured figure implied.

The EOP partials agree with a differenced rotation at 4.8e-9, 1.9e-9 and 1.7e-7. **All three are the difference's own roundoff floor, not the accuracy of the partials.** The third was swept over four decades of perturbation to establish that: it falls to 5.4e-10 at a 0.1 s step, and the product of error and step has no trend, scattering about 4.3e-11 — roughly 28 ulp in the differenced rotation. A step-independent error could not have fallen by 306 times.

6.4 The limit of every self-test

A wrong Euler sense, quaternion order or product still yields an orthogonal, unit-determinant matrix that round-trips through its own inverse. **No self-test can catch a consistently wrong toolbox.** The conventions here are therefore checked against independent authorities, and the convention struct travels with the data so that a mismatch between two objects is detectable. That is a statement about what has been checked, not a proof of correctness.

7 API reference

Generated from the help block of each function, grouped by the layers of `Contents.m`. Calls go downward only. For the full argument tables, the METHOD discussion and the worked examples, use `help referenceSystem.<name>` or the browsable function reference — this chapter is the map, not the territory.

L0 — Time and constants

Function	Summary
`toolboxVersion`	Version, build date and provenance of the toolbox.
`dataFolder`	Persistent data folder for installed EOP and kernels.
`iersConstants`	Named IERS, IAU and BIPM constants.
`ellipsoidParameters`	Defining and derived parameters of a reference ellipsoid.
`timeScale`	Registry of the supported time scales.
`normalizeEpoch`	Validate an epoch struct and put it in canonical form.
`mjdToEpoch`	Epoch from a decimal Modified Julian Date.
`daySecondToEpoch`	Epoch from an Nx2 day-number and seconds-of-day matrix.
`datetimeToEpoch`	Epoch from a MATLAB datetime array.
`gpsWeekToEpoch`	Epoch from a GPS week number and seconds of week.
`j2000SecondsToEpoch`	Epoch from seconds elapsed since the J2000.0 epoch.
`epochTo`	Epoch to one of the five external representations.
`addSeconds`	Advance an epoch by seconds of its own time scale.
`secondsBetween`	Seconds elapsed from one epoch to another.
`leapSecondTable`	The single leap-second authority of the toolbox.
`convertTimeScale`	Convert an epoch between any two supported time scales.
`fundamentalArguments`	Delaunay, planetary and Doodson arguments.

L1 — Earth orientation parameters

Function	Summary
`readEop`	Read an IERS Earth orientation series into the eop struct.
`interpolateEop`	Interpolate an EOP series to a vector of epochs.
`subdailyEopCorrection`	Sub-daily ocean tide and libration variation in the EOP.
`zonalTideEopCorrection`	Long-period zonal tide effect on Earth rotation.
`downloadEop`	Fetch a current IERS series, with local caching (optional).

L2 — Earth orientation transformation, ITRS <-> GCRS

Function	Summary
`cipCoordinates`	CIP coordinates and CIO locator, IAU 2006/2000A.
`earthOrientationInterpolant`	Chebyshev panels for the ITRS to GCRS chain.
`earthOrientationPartials`	Partials of the ITRS to GCRS rotation in the EOP.
`rotationItrsToGcrsPartials`	Partials of the ITRS-to-GCRS rotation w.r.t. EOP.
`evaluateInterpolant`	Evaluate one panel of an Earth orientation interpolant.
`writeInterpolant`	Write an Earth orientation interpolant to a file.
`readInterpolant`	Read an interpolant written by writeInterpolant.
`earthRotationAngleRate`	Rate of advance of the Earth rotation angle.
`earthRotationAngle`	Earth rotation angle and its rate of advance.
`greenwichSiderealTime`	Greenwich mean or apparent sidereal time.
`equinoxNutationMatrix`	Classical nutation-precession-bias matrix.
`rotationGcrsToCirs`	Celestial-to-intermediate rotation from the CIP and CIO.
`rotationTirsToItrs`	Polar motion rotation from the pole coordinates.
`rotationItrsToGcrs`	Terrestrial-to-celestial rotation and its derivatives.
`applyRotation`	Apply a rotation and its derivatives to a state.
`invertRotation`	Inverse of a rotation and of its time derivatives.

L3 — Terrestrial frame realizations

Function	Summary
`itrfParameters`	Published Helmert parameters from ITRF2020 to a past ITRF.
`convertItrf`	Convert positions and velocities between ITRF realizations.
`applyHelmert7`	Seven-parameter similarity transformation, exact both ways.
`applyMolodensky`	Molodensky datum transformation, a legacy compatibility path.

L4 — Coordinate representations

Function	Summary
`geodeticToCartesian`	Ellipsoidal geodetic coordinates to Cartesian coordinates.
`cartesianToGeodetic`	Cartesian coordinates to ellipsoidal geodetic coordinates.
`cartesianToEnu`	Earth-fixed Cartesian to local East-North-Up coordinates.
`enuToCartesian`	Local East-North-Up to earth-fixed Cartesian coordinates.
`enuToNed`	Local East-North-Up to local North-East-Down coordinates.
`nedToEnu`	Local North-East-Down to local East-North-Up coordinates.
`rotationEcefToEnu`	Earth-fixed to local East-North-Up rotation.

`rotationEcefToNed`	Earth-fixed to local North-East-Down rotation.
`enuToAzimuthElevation`	Local East-North-Up to azimuth, elevation and range.
`azimuthElevationToEnu`	Azimuth, elevation and range to local East-North-Up.
`cartesianToSpherical`	Cartesian to geocentric spherical coordinates.
`sphericalToCartesian`	Geocentric spherical to Cartesian coordinates.
`geodeticToUtm`	Geodetic coordinates to Universal Transverse Mercator.
`utmToGeodetic`	Universal Transverse Mercator to geodetic coordinates.
`geoidUndulation`	Interpolate the geoid undulation from a regular grid.
`ellipsoidalToOrthometric`	Ellipsoidal height to orthometric height.
`orthometricToEllipsoidal`	Orthometric height to ellipsoidal height.

L4b — Rotation algebra

Function	Summary
`attitudeConvention`	Build or validate the rotation-algebra convention struct.
`rotationX`	Elementary rotation about the x axis.
`rotationY`	Elementary rotation about the y axis.
`rotationZ`	Elementary rotation about the z axis.
`rotationFromEulerSequence`	Rotation matrix from an Euler angle sequence.
`eulerSequenceFromRotation`	Euler angles of a rotation, all twelve sequences.
`rotationFromRollPitchYaw`	Rotation matrix from roll, pitch and yaw.
`rollPitchYawFromRotation`	Roll, pitch and yaw of a rotation.
`rotationFromQuaternion`	Rotation matrix of a unit quaternion.
`quaternionFromRotation`	Unit quaternion of a rotation, by Shepperd's method.
`rotationFromAxisAngle`	Rotation matrix from an axis and an angle, Rodrigues.
`axisAngleFromRotation`	Rotation axis and angle of a rotation matrix.
`quaternionFromEulerSequence`	Quaternion from an Euler angle sequence.
`eulerSequenceFromQuaternion`	Euler angles of a quaternion, twelve sequences.
`multiplyQuaternions`	Product of two quaternion series, conventions compared.
`invertQuaternion`	Inverse of a unit quaternion, its conjugate.
`normalizeQuaternion`	Unit norm and a consistent sign for a quaternion series.

L5 — Orbit, attitude and navigation

Function	Summary
`rotationGcrsToRsw`	Inertial to orbit frame rotation from a satellite state.
`interpolateQuaternions`	Interpolate an attitude series to a vector of epochs.
`rotationNavToBody`	Navigation-frame to body-frame rotation from roll, pitch and yaw.
`angularVelocityToRotationRate`	Rotation rate from an angular velocity.
`angularVelocityFromRotationRate`	Angular velocity from a rotation rate.
`integrateAttitude`	Propagate attitude over a gyro rate series.
`integrateAttitudeIncrements`	Propagate attitude over integrated angle increments.
`wanderAzimuthRotation`	Wander-azimuth local-level mechanisation.
`readImu1b`	Read a GRACE-FO IMU1B filtered-gyro-angle product.
`graceFoImuAlignment`	GRACE-FO gyro axes and IMU-to-Satellite-Frame rotation.
`graceFoImuIncrements`	Satellite-Frame angle increments from GRACE-FO IMU1B.
`ephemerisBodies`	Bodies a provider supplies, with their accuracy tier.
`barycentricBodyState`	Position and velocity of a body in the BCRS.
`geocentricBodyState`	Position and velocity of a body seen from the Earth.
`readEphemeris`	Furnish NAIF kernels and record what was furnished.
`downloadKernels`	Fetch the SPICE kernels the MICE provider needs, or list them.
`gcrsToBcrs`	Express a geocentric vector in barycentric coordinates.
`bcrsToGcrs`	Express a barycentric-coordinate vector in the GCRS.
`bodyRotationElements`	IAU WGCCRE rotational elements of the Sun and planets.
`rotationIcrfToBodyFixed`	Rotation from the ICRF to a body-fixed frame.

L6 — Dispatch and infrastructure

Function	Summary
`convertCoordinates`	Single point of access across the L3 and L4 transformations.
`mexInfo`	State of the MEX gateway and the leap-second authority.
`mexSourceDigest`	Content digest of every C source the gateway uses.
`buildMex`	Compile the package-private MEX gateway.
`buildDocumentation`	Generate the function reference and the search database.
`runTestSuite`	Run the test suite and write one plain-text report.
`teardownReferenceSystem`	Release the toolbox so its folder can be deleted.

7.1 Accuracy statements, in full

The ACCURACY block is mandatory in this toolbox: every approximation is named and quantified where the function that makes it can be read. They are reproduced here so that the manual carries the same statements the code does, and cannot drift from them.

8 Where to look next

Document	What it holds
`HANDOVER_referenceSystem.md`	**The single point of truth.** Design, decisions D1–D110, standing obligations O1–O31, the revision log and the open-items register
`BEST_PRACTICE.md`	The lessons this project paid for, extracted as a template for future work
`doc/referenceSystem_functionReference.html`	Every function, every argument, generated
`doc/referenceSystem_accuracy.html`	The full error budget
`doc/referenceSystem_ephemerides.html`	Ephemeris tiers, BCRS, body frames, EOP partials
`doc/referenceSystem_cApi.html`	The public C interface and the interpolant file format
`referenceSystem_testReport.txt`	The last run: benchmarks, convergence, notes, failures

referenceSystem v2.1.209 — developed by Matthias Weigelt with the assistance of Claude Opus 5 (Anthropic). Built 2026-08-21.